

POKER-REACT-NODE: ONE-PAGE APP SUMMARY

WHAT IT IS

A full-stack Texas Hold'em app with a React + Redux frontend and Node/Express backend.
It includes game actions, player seating, real-time events, and MySQL-backed user/session data.

WHO IT'S FOR

Primary persona: Two players in a heads-up Texas Hold'em game (docs/mvp_make_it_work.md).
Secondary persona: Developers learning collaborative TypeScript, TDD, and backend/frontend practices (docs/project_learning_objectives.md).

WHAT IT DOES

- Creates accounts and signs users in (users.create, users.signin, users.getProfile).
- Lets users join/leave table seats and fetch current seat state.
- Supports core actions: fold, check, call, and bet.
- Fetches and renders game state: seats, hole cards, acting seat, pot, and community cards.
- Pushes real-time updates over Socket.IO (player_joined, player_left, start_game, deal_cards, update_pot).
- Persists users, tables, and sessions in MySQL schema tables.
- External winning-hand API is listed as MVP intent but Not found in repo implementation.

HOW IT WORKS (ARCHITECTURE)

Components/services

- Frontend: Vite React app, Redux slices, Router pages, FetchFacade, socket hook.
- Backend API: Express server with session middleware and auto-wired handlers from shared APIMethod metadata.
- Game domain: PokerTable, Game, Dealer, Player, Seat, Deck.
- Real-time layer: SocketServer + Rooms + Sockets + GameEmitter.
- Data layer: MySql wrapper + SessionStore -> users, poker_tables, poker_table_users, sessions, users_sessions.

Data flow (compact)

- 1) Frontend POSTs /api/{method} (for example games.bet).
- 2) Router resolves handler from generated APIMethod map.
- 3) Handler calls domain/services (GameLobbyService, Dealer, UserService, PokerTable).
- 4) Domain emits events via GameEmitter -> Rooms/Sockets -> clients.
- 5) Frontend socket subscribers dispatch Redux updates and rerender UI.

HOW TO RUN (MINIMAL GETTING STARTED)

- 1) From repo root: npm run install:all
- 2) Create backend/.env with DB_HOST, DB_USER, DB_PASSWORD, DB_DATABASE=dev_poker
- 3) Create database/schema: npm run create-dev-db
- 4) Start app: npm run start:dev
- 5) Open http://localhost:5173 (backend: http://localhost:3000)

NOT FOUND IN REPO

- Production deployment/configuration guidance.